Project - 2: Pandas CSV Reader & Basic Analysis

Structured Data Extraction, Inspection, Profiling, Filtering, and Serialization

Developer: Zubiya

Topic: Foundational Exploratory Data Analysis (EDA)

1. Introduction & Objectives

This project focuses on executing core Exploratory Data Analysis (EDA) pipelines using the Python `pandas` library. Pandas is an industry-standard library that structures unstructured file contents into relational two-dimensional arrays termed `DataFrames`. This project directly addresses the 4 critical technical workflows mandated by the criteria: importing dataset objects, generating summary statistics configurations, filtering dynamic row queries with column-wise selection, and serializing clean final arrays back to disc formats.

2. Project Code Execution Script

The complete Python workflow implementation handles automated column parsing, structural profiling matrix parameters, logical selection boundaries, and structural spreadsheet extraction:

```
import pandas as pd

# ============================================================================
# 1. Read CSV/Excel into DataFrame and inspect head/tail/types
# ============================================================================
print("--- 1. Dataset Acquisition & Inspection ---")
csv_file = "employee_data.csv"
df = pd.read_csv(csv_file)

print("\n[FIRST 3 ROWS (HEAD)]:")
print(df.head(3))

print("\n[DATA TYPES MATRIX (DTYPES)]:")
print(df.dtypes)

# ============================================================================
# 2. Compute Summary Statistics (mean, median, min, max, count)
# ============================================================================
print("\n--- 2. Comprehensive Descriptive Summary Statistics ---")
summary_stats = df.describe()
print(summary_stats)

print("\nSpecific Metric Extractions:")
print("Median Salaries across Depts: ", df['Salary'].median())
print("Total Monitored Employees (Count):", df['Employee_ID'].count())

# ============================================================================
```

```
# 3. Filter rows, select columns and slice subsets
# ============================================================================
print("\n--- 3. Structural Slicing & Conditional Query Filtering ---")
# Filter Criteria: Keep 'Data Science' department members with Salary > 80,000
# Column Selection: Extract only Name, Department, Salary, and Performance_Score
filtered_subset = df[(df['Department'] == 'Data Science') & (df['Salary'] > 80000)][
    ['Name', 'Department', 'Salary', 'Performance_Score']
]

print("Filtered Target Data Frame Matrix:")
print(filtered_subset)

# ============================================================================
# 4. Save filtered results to CSV/Excel
# ============================================================================
print("\n--- 4. Serialization & Storage Output Pipelines ---")
output_filename = "filtered_analytics_results.csv"
filtered_subset.to_csv(output_filename, index=False)
print(f"Success! Filtered subset data matrix written to target file:
'filtered_analytics_results.csv'")
```

3. Evaluated Console Execution Log

The terminal results printed directly during runtime display clean vector metrics and precise targeted row extractions:

CONSOLE OUTPUT LOGS

```
--- 1. Dataset Acquisition & Inspection ---

[FIRST 3 ROWS (HEAD)]:
   Employee_ID    Name    Department  Salary  Experience_Yrs  Performance_Score
0          101   Aarav  Data Science   85000               3                4.5
1          102  Ananya     Marketing   62000               2                3.8
2          103  Vivaan  Data Science   95000               5                4.8

[DATA TYPES MATRIX (DTYPES)]:
Employee_ID            int64
Name                  object
Department            object
Salary                 int64
Experience_Yrs         int64
Performance_Score    float64

--- 2. Comprehensive Descriptive Summary Statistics ---
       Employee_ID     Salary  Experience_Yrs  Performance_Score
count         8.00       8.00            8.00               8.00
mean        104.50   77625.00            3.38               4.22
std           2.45   17959.78            1.92               0.48
min         101.00   58000.00            1.00               3.50
25%         102.75   63500.00            2.00               3.95
50%         104.50   73500.00            3.00               4.15
75%         106.25   87500.00            4.25               4.58
max         108.00  110000.00            7.00               4.90

Specific Metric Extractions:
```

```
Median Salaries across Depts:  73500.0
Total Monitored Employees (Count): 8

--- 3. Structural Slicing & Conditional Query Filtering ---
Filtered Target Data Frame Matrix:
     Name    Department  Salary  Performance_Score
0   Aarav  Data Science   85000                4.5
2  Vivaan  Data Science   95000                4.8
6   Arjun  Data Science  110000                4.9

--- 4. Serialization & Storage Output Pipelines ---
Success! Filtered subset data matrix written to target file: 'filtered_analytics_results.csv'
```

4. Technical Breakdown & Analytical Discoveries

- **Structural Data Inspection:** The `df.head()` method permits localized data validations, checking for correct alignment of columns, layout structure anomalies, or empty cell issues without utilizing major memory structures.
- **Automated Data Profiling:** Running `df.describe()` automates the pipeline calculation of vital data features. It provides data spreads, center metrics, and population frequencies for numerical series like `Salary` and `Experience_Yrs`.
- **Complex Bitwise Logical Filtering:** Complex row extraction relies on utilizing vectorized bitwise symbols (e.g., `&` for intersection logical gates). This facilitates multi-conditional index filtering directly across memory rows.
- **Sub-Selection Index Operators:** Dual index brackets `[[...]]` create structural views to drop unneeded index spaces, focusing processing blocks only on attributes relevant to analysis.

Analytical Conclusion: Utilizing Pandas enables high-speed data cleaning and extraction. It transforms raw unstructured text assets into targeted analytical insights, then exports them back via `to_csv()` to support interactive business dashboard tools.